T1 贝贝的幸运号码（lucky）

枚举数位平方和，反推出号码

等式 $n = kf(n)$ 中，k 已知。只要确定 $f(n)$，号码 n 就随之确定。因此可以枚举数位平方和，而不必枚举可能达到 10^{18} 的号码。

设 b 的十进制位数为 D。区间中的正整数至多有 D 位，每位数字的平方至多为 81，所以

$$1 \le f(n) \le 81D.$$

完整范围内 $D \le 19$，候选平方和至多只有 1539 个。依次枚举 $t = 1, 2, \ldots, 81D$，令 $x = kt$。当且仅当 $a \le x \le b$ 且 $f(x) = t$ 时，把答案加一。

这里必须重新计算 $f(x)$：枚举的 t 只是一个猜测，乘积 kt 的数位平方和未必等于它。任何合法号码都会在枚举到自己的平方和时被找到；又因为 $k > 0$，不同的 t 对应不同的 x，不会重复计数。

实现时先判断 $t > b/k$。若成立，则当前及之后的乘积都超过 b，可以结束枚举；否则再计算 $k \times t$，保证乘积不超过 10^{18}，避免先乘法后判断造成溢出。区间端点用非严格比较保留，$a = b$ 也无需特殊处理。

共枚举 $81D$ 个候选，每次计算数位平方和需要 $O(D)$ 时间，总时间为 $O(81D^2)$，额外空间为 $O(1)$。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

int calc(long long x){
    int sum = 0;
    while(x > 0){
        int d = x % 10;
        sum += d * d;
        x /= 10;
    }
    return sum;
}

int main(){
    long long k,a,b;
    cin >> k >> a >> b;
    int d = 0;
    for(long long x = b; x > 0; x /= 10) d++;

    int ans = 0;
    for(int t = 1; t <= 81 * d; t++){
        if(t > b / k) break;
        long long x = k * t;
        if(x >= a && calc(x) == t) ans++;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

以下分值按原题的测试点组列出。令 $L = b - a + 1$，令 $M = \max(0, \min(b, 1539k) - a + 1)$；计算上界时须先判断乘法是否会超过 b。第 2 项的同一程序覆盖前三档，共 80 分，不重复相加。

序号	条件与点组	点组分值	做法与复杂度
1	$1 \sim 8$：$k, a, b \le 10^6$	40 分	枚举区间中的每个整数，计算平方和并检查等式。用 $x \bmod k = 0$ 且 $x/k = f(x)$ 判断，避免乘法溢出。时间 $O(LD)$，空间 $O(1)$。
2	$9 \sim 12$：$k \le 1000$	20 分	只枚举到 $\min(b, 1539k)$，先检查 x 是否为 k 的倍数，再计算平方和。此档区间长度 $M \le 1539000$，而需拆位的倍数总共至多 1539 个。时间 $O(M + 1539D)$，空间 $O(1)$。
复用 2	$13 \sim 16$：$1 \le b - a \le 5 \times 10^7$	20 分	复用上一行程序。此档 $M \le 50000001$，仍至多拆位 1539 次，复杂度不变。它同时覆盖第 1 档，因此前三档合计保证 80 分。
—	$17 \sim 20$：无附加限制	20 分	枚举平方和并反推号码，最多 1539 个候选，按正文检验。时间 $O(81D^2)$，空间 $O(1)$，覆盖全部测试点。

独立源码为 lucky_1.cpp、lucky_2.cpp。若进一步直接枚举这些倍数的编号 t，便得到正文的满分做法。

T2 贝贝的守护行动（guard）

把守卫的实际收益分批排序

目标是尽量多地击退小怪，因此应比较“再安排一名守卫，实际能多击退多少只”，而不能只比较 k_i。例如，某段只有 5 只小怪，即使 $k_i = 100$，一名守卫也只能产生 5 的收益。

对第 i 段防线，令

$$q_i = \left\lfloor \frac{a_i}{k_i} \right\rfloor, \qquad r_i = a_i \bmod k_i.$$

前 q_i 名守卫每人都能击退 k_i 只小怪；若 $r_i > 0$，再安排一人能击退 r_i 只；之后收益为 0。所以只需保存两批收益：

- 单次收益为 k_i，共有 q_i 次；
- 单次收益为 r_i，共有 1 次。

这样，每段至多生成两条记录，不必把可能多达 10^9 名守卫逐个展开。代码中 b[i].x 表示单次收益，b[i].y 表示该收益可以使用的次数。次数为 0 或收益为 0 的记录保留也不影响答案。

将全部记录按单次收益从大到小排序，依次尽量选取。这样做不会违反同一条防线的安排顺序：余数收益 $r_i < k_i$，若选到了它，该防线前面的完整收益已经选完；相同收益之间的先后次序不影响结果。因此，收益最大的若干次安排能够组成合法分配。任意其他方案若选了较小收益而漏选较大收益，交换后击退总数不会减少，故按降序选取能使击退总数最大。

处理一条记录时，取

$$\mathit{take} = \min(s, \text{次数}),$$

再令 s 减去 *take*。这条记录未被选中的收益总量为

$$\text{单次收益} \times (\text{次数} - \mathit{take}).$$

两批收益的总量恰好为 $q_i k_i + r_i = a_i$，所以把全部未选收益相加，就是剩余小怪数。若所有小怪已经被击退，剩余守卫放到任意防线上即可，不会再改变答案。

共 $2n$ 条记录，排序需要 $O(n \log n)$ 时间，扫描需要 $O(n)$ 时间；总时间 $O(n \log n)$，空间 $O(n)$。小怪总数最多为 10^{14}，收益乘积和答案均使用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
struct node{
    long long x,y;
}b[2 * N];

bool cmp(node u,node v){
    return u.x > v.x;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    long long s;
    cin >> n >> s;
    int cnt = 0;
    for(int i = 1; i <= n; i++){
        long long a,k;
        cin >> a >> k;
        b[++cnt] = {k,a / k};
        b[++cnt] = {a % k,1};
    }
    sort(b + 1,b + cnt + 1,cmp);

    long long ans = 0;
    for(int i = 1; i <= cnt; i++){
        long long take = min(s,b[i].y);
        ans += b[i].x * (b[i].y - take);
        s -= take;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

本题有依赖计分。子任务 $1,2,3$ 的公共上限为 $n \le 100$、$s \le 10000$、$a_i \le 100$。第 2 项程序自然覆盖这三个子任务，满足其全部依赖，共保证 $17+21+23=61$ 分。

序号	条件与子任务	子任务分值	做法与复杂度
1	子任务 1：$k_i = 1$	17 分	每名守卫只要遇到尚未清空的防线，就能击退一只小怪，答案为 $\max(0, \sum a_i - s)$。时间 $O(n)$，空间 $O(1)$。
2	子任务 2：$k_i \le 2$，依赖 1	21 分	逐名安排守卫，每次扫描所有防线，选择当前 $\min(a_i, k_i)$ 最大者，更新该段剩余小怪数。没有正收益时结束。设 $T = \sum \lceil a_i/k_i \rceil$，时间 $O(n(1+\min(s,T)))$，空间 $O(n)$。
复用 2	子任务 3：$k_i \le 100$，依赖 $1,2$	23 分	复用上一行程序，单次收益仍按实际剩余数量计算。公共范围下至多扫描 10000 轮，每轮至多 100 段；连同前两档保证 61 分。
—	子任务 4：完整范围，依赖 $1,2,3$	39 分	把重复收益合并为批次，再按收益降序选取。时间 $O(n \log n)$，空间 $O(n)$，通过全部依赖后得到 100 分。

独立源码为 guard_1.cpp、guard_2.cpp；子任务 $2,3$ 复用后一份程序。逐名贪心使用的正是正文中的收益顺序，只是没有合并重复操作。

T3 贝贝的合影安排（photo）

排序后，寻找包含每个班的最窄区间

所选同学的最高、最低身高决定答案，其余人的身高只需落在两者之间。因此，把所有同学按身高排序，并保留各自的班级编号，就可以把问题转为寻找一个身高区间，使它包含每个班的至少一名同学。

这个转化保留了最优答案：任何合法合影方案的最低、最高身高之间都包含全部班级；反过来，任何覆盖全部班级的区间，都可以从每班选出一人，其身高差不超过该区间宽度。两边分别给出界限，所以最小合影身高差恰好等于最窄覆盖区间的宽度。

令总人数为 $N = ns$，排序后的同学记为 $a_1, a_2, \ldots, a_N$。窗口 $[l, r]$ 的宽度为 $a_r.h - a_l.h$。右端点向右移动时，窗口只会增加同学，已经覆盖的班级不会消失；因此用双指针维护窗口。

为避免每次重新统计整个窗口，维护 cnt[id]，表示窗口内第 id 个班的人数；再维护 covered，表示人数非零的班级数：

- 加入右端点同学时，若该班人数原为 0，则 covered 加一；随后增加该班人数。
- 当 covered == n 时，先用当前宽度更新答案，再删除左端点同学并右移 l；若其班级人数减为 0，则 covered 减一。
- 只要窗口仍覆盖全部班级，就继续收缩左端点。

固定右端点时，左端点越靠右，区间越窄，所以应一直收缩到首次失去覆盖。已经删除的左端点也无需在之后恢复：它在较早的右端点处已经形成过合法区间，未来右端点更靠右，使用同一个左端点只会使宽度更大，不可能改善当时的答案。这样既不会漏掉更优区间，也使两个指针都只向右移动。

每位同学最多加入、删除各一次，窗口部分共 $O(N)$ 次操作，且每次班级统计均为 $O(1)$。连同排序，总时间为 $O(N \log N)$，空间为 $O(N + n)$。完整范围 $N \le 10^6$；身高差不超过 $10^9 - 1$，使用 int 足够。一个班、重复身高等情况都由相同流程处理，最小值可以是 0。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 1000000 + 10;
const int M = 1000 + 10;
struct node{
    int h,id;
}a[N];
int cnt[M];

bool cmp(node u,node v){
    return u.h < v.h;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n,s;
    cin >> n >> s;
    int tot = 0;
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= s; j++){
            int h;
            cin >> h;
            a[++tot] = {h,i};
        }
    }
    sort(a + 1,a + tot + 1,cmp);

    int l = 1,covered = 0,ans = 1000000000;
    for(int r = 1; r <= tot; r++){
        int id = a[r].id;
        if(cnt[id] == 0) covered++;
        cnt[id]++;
        while(covered == n){
            ans = min(ans,a[r].h - a[l].h);
            id = a[l].id;
            cnt[id]--;
            if(cnt[id] == 0) covered--;
            l++;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

本题的子任务独立计分。仍记 $N = ns$。第 2 项方法也能处理 $n = 2, s \le 1000$，至少覆盖 $11 + 22 = 33$ 分；第 3 项方法覆盖前四个子任务，至少覆盖 75 分。这些是保证覆盖的下界，不表示算法在更高子任务上必然超时。

序号	条件与子任务	子任务分值	做法与复杂度
1	子任务 1：$n = 2$	11 分	两个班分别排序，用两个指针比较当前身高差，每次移动较小身高所在的指针。它与另一班后续更大身高配对不会更优，因此可舍弃。时间 $O(s \log s)$，空间 $O(s)$。
2	子任务 2：$n, s \le 100$	22 分	所有人排序后，枚举左端点，每次从该处重新向右扩展，增量记录各班人数，首次覆盖全部班级时更新答案。时间 $O(N \log N + N(N + n))$，空间 $O(N + n)$；在第 1 档 $N \le 2000$，也能通过，合计保证 33 分。
3	子任务 3：$n, s \le 250$	9 分	使用正文的双指针维护人数，但每次通过扫描全部 n 个班判断是否已覆盖。一次判断 $O(n)$，判断次数 $O(N)$，总时间 $O(N \log N + Nn)$，空间 $O(N + n)$。
复用 3	子任务 4：$n, s \le 500$	33 分	复用上一行程序。此档 $Nn \le 1.25 \times 10^8$，不必重新枚举左右端点；它也覆盖前三档，至少覆盖 75 分。
—	子任务 5：完整范围	25 分	额外维护 covered，把每次覆盖判断降为 $O(1)$。总时间 $O(N \log N)$，空间 $O(N + n)$，覆盖全部子任务。

独立源码为 photo_1.cpp、photo_2.cpp、photo_3.cpp；子任务 $3, 4$ 复用最后一份。第 2 项中，固定左端点后只需取第一个合法右端点，再向右扩展只会使宽度不减。

T4 贝贝的双人挑战（pair）

按可选组连通性，分别考虑无法配对的人数

每名同学只能在两个活动组之间选择。这使两个组通过这名同学发生联系：把活动组看作点，把同学看作连接 a_i, b_i 的一条无向边；把同学分到某个组，就是把这条边交给其中一个端点。两种选择相同时是一条自环，多名同学有相同选择时保留多条边。

不同连通块之间没有同学可以跨过去，因此可以分别安排。设一个连通块共有 E 条边，也就是共有 E 名同学。无论怎样分组，每支双人队都消耗两人，所以该块至少剩下 $E \bmod 2$ 人。接下来说明这个下界一定可以达到。

在连通块中任选一棵生成树和一个根。先把所有非树边及自环对应的同学，任意分给其允许的一个端点；树边暂不安排。然后从树的叶子向根处理：

- 处理非根节点 v 时，它与子节点之间的树边已经安排完，唯一尚未安排的是连接父亲的那条树边。
- 若 v 当前人数为奇数，就把这条边对应的同学分到 v；若为偶数，就分到父亲。

这样 v 的最终人数总为偶数，且之后不会再改变。处理完所有非根节点后，只有根可能为奇数；整块总人数为 E，其余组均为偶数，所以根的人数奇偶性恰好为 $E \bmod 2$。因此每块最少剩下 0 或 1 人，取决于该块的**边数**是否为奇数，而不是点数是否为奇数。只有一个活动组时，所有自环已在第一步安排，这个结论也成立。

题目只要求积分，不必真的构造上述分组。用并查集维护活动组所在的连通块，并在根节点上记录 cnt，表示块中的同学数：

- 读入一名同学的两个选择，找到两端的根 x, y。
- 若两根不同，合并两个集合，并把它们已有的同学数相加。
- 无论两根原来是否相同，都给合并后的根增加一名同学。

最后统计 cnt 为奇数的根的数量 c，答案就是

$$k - c.$$

自环虽然两端相同，也只代表一名同学，计数只加一；空组对应没有边的块，不扣分。答案可以为负数。代码按集合大小合并并进行路径压缩，时间为 $O((n+m)\alpha(m))$，其中 α 是反阿克曼函数；空间为 $O(m)$。证明中的生成树只用于说明存在性，程序无需建图或递归遍历整张图。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
int fa[N],sz[N],cnt[N];

int find(int x){
    if(fa[x] == x) return x;
    return fa[x] = find(fa[x]);
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n,m;
    long long k;
    cin >> n >> m >> k;
    for(int i = 1; i <= m; i++){
        fa[i] = i;
        sz[i] = 1;
    }
    for(int i = 1; i <= n; i++){
        int a,b;
        cin >> a >> b;
        int x = find(a),y = find(b);
        if(x != y){
            if(sz[x] < sz[y]) swap(x,y);
            fa[y] = x;
            sz[x] += sz[y];
            cnt[x] += cnt[y];
        }
        cnt[x]++;
    }
    long long ans = k;
    for(int i = 1; i <= m; i++){
        if(fa[i] == i && cnt[i] % 2 == 1) ans--;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

每个测试点 5 分，点组独立计分。第 1 项搜索对相同选择只走一次分支，因此在全部 $a_i = b_i$ 时也能线性完成，自然覆盖前 10 点，共保证 50 分。第 2 项是固定去向时不使用递归的直接统计法，保证 10 分；二者的覆盖不能重复相加。

序号	条件与点组	点组分值	做法与复杂度
1	$1 \sim 8$：$n, m \le 20$	40 分	搜索每名同学的两个去向。加入或撤销一个人时，只更新所在组的人数及奇数组数；到叶子时取奇数组数的最小值。搜索节点数 $O(2^n)$，总时间 $O(2^n + n + m)$，空间 $O(n + m)$，递归深度 n。
2	$9 \sim 10$：所有 $a_i = b_i$	10 分	每人去向唯一，直接统计各组人数，从 k 中扣去奇数组数。时间 $O(n + m)$，空间 $O(m)$。
—	$11 \sim 20$：无附加限制	50 分	并查集统计连通块中的边数，对每个奇数边块扣一分。时间 $O((n + m)\alpha(m))$，空间 $O(m)$，覆盖全部测试点。

独立源码为 pair_1.cpp、pair_2.cpp。搜索中若两种选择相同，只走一次分支；全部选择相同时只访问 $n + 1$ 层，时间为 $O(n + m)$、递归深度为 n，需注意栈空间。撤销时恢复该组人数和原来的奇数组数。